I teach computer science as a design discipline: given an under-specified problem, working out which constraints actually matter and reasoning about the trade-offs among approaches that satisfy them. Underneath that sits a prior question: what would make a solution correct at all? My goal is for students to reason systematically through unfamiliar problems rather than stumble onto working solutions, and to keep doing so without an external requirement. Generative AI sharpens this, not because it makes these skills less valuable but because it removes the difficulty that used to force students to develop them: when a passable answer costs almost nothing, nothing in the task itself requires the reasoning the task was meant to teach. That reasoning was never visible in a working program to begin with, which is why it has to be designed into what a course asks students to produce and into how a classroom runs.

Curricular Contributions and Course Development

Reasoning about trade-offs and correctness is a core skill for a student of computer systems. My background is at the hardware–software interface, so I can teach broadly across computer systems, from Algorithms, Digital Logic to Parallel Computing, Compilers and Computer Architecture. Across these topics, the focus is on identifying and analyzing the relevant trade-offs.

That priority shapes how I would teach high-performance computing in particular. Rather than treating optimizations as static techniques to memorize, students would analyze how specific transformations affect execution cost while preserving program semantics, returning to a single question: what are the benefits of this choice, and what does it cost on this machine? In Embedded Machine Learning, students would co-design models from training through deployment on resource-constrained hardware.

I am also keen to develop an interdisciplinary computational design studio for students applying computing within another field. Students would translate questions from fields outside computer science into computational specifications, articulate assumptions, and develop rubrics for evaluating solutions. They could then prototype with AI-assisted tools, while being assessed on their ability to defend and revise those specifications. Rather than requiring students to build a foundation in CS before beginning a project, I would introduce fundamental concepts as their projects reveal why they are needed.

At the introductory level, my main interest is how formal methods are taught within introductory classes. Contracts, invariants, and reasoning about correctness appear throughout curricular guidelines, but not usually in introductory sequences. CMU's Principles of Imperative Computation demonstrates that formal reasoning can be taught alongside data structures at scale. I want to introduce formal reasoning earlier: students stating on paper what a piece of code should do and what must hold as it runs before implementing it, so that the gap between the paper reasoning and the running program becomes the object of study. This requires no specialized language or additional course slot, which makes it something I could test within a course I was already teaching.

Teaching Framework: Dialogue, Practice, and Reflection

I organize much of my teaching around a dialogue–practice–reflection (DPR) cycle, an adaptation of Kolb's experiential learning cycle. Each learning unit begins with *dialogue*, in which students and the instructor articulate assumptions, constraints, and possible approaches. Students then move into *practice*, where they have to apply those ideas to a concrete problem and encounter complications that were invisible during the initial discussion. *Reflection* closes the cycle by asking what worked, what failed, which assumptions proved important, and what lessons can be transferred to the next problem.

Over seven offerings of *How to Write Fast Code I*, a 100-student course on high-performance computing, I have served as a teaching assistant and guest lecturer and provided feedback on course materials. The problem I kept returning to was how to see students' reasoning rather than just their answers. In class, students worked through small-group exercises on material they had just encountered, then walked through solutions together so that common mistakes surfaced while the reasoning behind them was still fresh; an external observation of one of my lectures noted students volunteering to work through examples and asking follow-up questions unprompted. When the course went remote and hybrid, a shared workspace where each group recorded its analysis on a dedicated slide kept that reasoning visible to in-person and remote students alike, and it became a resource students returned to for exams and projects.

In *Computational Problem Solving for Engineers*, a 40-student survey of supercomputing and computational sciences, the cycle operated over a homework period instead of a single class. Recitations opened with a discussion of the assignment's goals, which I synthesized into a short explanation of possible approaches. Implementation outside of class provided the practice. Reflection occurred through office-hour conversations and structured questions in homework reports, where students connected unexpected behavior back to their design decisions. Patterns across reflections showed me what gaps needed shoring up before they were ready to move to the next level of design questions.

At the scale of semester-long research projects, DPR becomes the cadence of supervision itself. Early in a project I model how to frame research questions and design experiments. Over time, I deliberately withdraw that scaffolding until students begin defending their own experiments and critiquing alternatives.

Teaching Computer Science in the Generative-AI Era

I want students to take increasing responsibility for directing and evaluating their own learning. Generative AI makes that responsibility harder to avoid: when any part of a task can be delegated, deciding what to do yourself becomes an explicit choice rather than a default. In Computational Problem Solving for Engineers, I added a reflection question to each assignment asking students to explain the trade-offs behind their implementation decisions, making their reasoning part of the assessed work.

The amount of AI welcomed in the classroom depends on the objective. In an introductory CS course, tracing a loop or debugging a program is how students build the mental models they will

need to successfully create more complicated programs later. In-class work protects some of that practice by construction, but most of it has to happen outside the room, and I do not think policy alone addresses that. One approach I would want to try is using class time to build skepticism students can carry into unsupervised work: asking them to generate code that "looks good," testing it to find subtle errors, then formally checking it. Whether this exercise changes how students use AI tools, I want to measure instead of assume. In an interdisciplinary design studio, where problem formulation is the learning objective, I would encourage AI-assisted implementation while assessing students' specifications and ability to defend them.

In both settings, assessment should make students' reasoning visible. Regardless of class size, live assessments let me probe that reasoning, from brief in-class explanations and structured check-ins to longer oral defenses and design reviews. Whatever the format, students should be able to justify their approach, compare alternatives, and adapt when assumptions or constraints change.

Reflection on Teaching

Asking students to justify their design decisions commits me to the same standard for my own teaching: a choice about how to run a class is a hypothesis about what students will do, and it deserves evidence rather than confidence. One of my most useful teaching experiences began with an intervention that failed. In *Computational Problem Solving for Engineers*, recitations traditionally included a video that provided context for the next assignment. I moved the video outside the classroom to free up recitation time for richer discussion, but students simply did not watch it and arrived to class without questions.

Rather than cut recitation short, as a class, we worked out the context the video provided – say, using convolutions to multiply large numbers – at the beginning of recitation. Once students had helped frame the problem, they asked more questions and class time was used to discuss possible implementation decisions and their trade-offs. The new recitation structure put students ahead of schedule on assignments and created time to explore related topics from students' own research. Students reported that they preferred shared context building over watching a video alone.

This experience reinforced a habit of evaluating teaching interventions through evidence of student learning rather than instructor intent. To this end, I have asked pedagogy experts from the University Teaching Center to observe my teaching, and used their suggestions to revise classroom activities. To prepare myself to teach large undergraduate classes, I took a course, Evidence-Based Teaching in STEM. In my subsequent recitations, I have used regular in-class assessments and direct student feedback to improve my teaching throughout the semester.

Teaching is a discipline that requires intentional practice, not an innate talent. The habits I want students to leave with are narrow enough to state plainly: ask what a solution is supposed to guarantee, ask how you would know that it does, and ask what it costs. A student who does that reliably can evaluate systems – their own, a classmate's, or one a tool produced – and recognize when a plausible answer is the wrong one.